Phân tích RQ1 – Separate Chaining vs Linear Probing (lp-tomb)

Nhiệm vụ C1 – kết quả chạy RQ1 (Thứ Sáu 23/10) + bản nháp phân tích và Discussion

0. Trạng thái các mục của ngày 23/10

Mục	Trạng thái
Thí nghiệm A: α = 0,50; 0,75; 0,90; 0,95 tại M = 2^{20}, sc và lp-tomb, (có mặt, văng mặt, dựng bảng; số lần dò và thời gian ở hai lượt riêng)	Xong
Thí nghiệm B: M = 2^{18}; 2^{20}; 2^{21}; 2^{22}; 2^{23} tại α = 0,75	Xong
Bộ dồn cụm: α = 0,75, M = 2^{18} và 2^{20}, 1 lượt	Xong
Đối chiếu số lần dò với bảng lý thuyết mục 3	Xong (mục 3 bên dưới). Tôi dùng công thức Knuth chuẩn vì chưa có file mục 3 của đề tài, bạn hãy đối chiếu lại với bảng gốc
`output_check` giữa sc và lp-tomb cùng seed	**22/22 cặp trùng nhau** (kèm `input_sha256` trùng)

Lưu ý quan trọng trước khi đưa vào paper: toàn bộ số liệu bên dưới chạy trên máy ảo 1 vCPU, không phải máy lab. Số lần dò mang tính xác định (không phụ thuộc máy) nên dùng được ngay. **Thời gian (ns, ms) và heap phụ thuộc máy, cần chạy lại trên PC lab** bằng đúng `rq1_jobs.txt` kèm `--machine-id` thật. Trong CSV giao kèm, tôi đã đổi `machine_id` thành `SANDBOX` để không bị nhầm với số đo của lab.

1. Thiết lập

- Cấu hình: seed = 42, giá trị mọi khóa = 1, τ = ∞ (lp-tomb không rebuild; RQ1 không có xóa nên không có tombstone).
- Lượt **count** và lượt **time** chạy riêng. Mỗi mốc chạy trong một JVM riêng (`-Xmx2g`) qua `BenchmarkWrapper` , tuần tự, timeout 120 s (cả 44 mốc `ok` , không mốc nào timeout).

- `BenchmarkRunner` chỉ đo `get` trên các khóa **có mặt**. Để có số liệu vắng mặt và dựng bảng, tôi viết thêm driver `RQ1Analysis` (không sửa file nào của nhóm): 1 triệu truy vấn có mặt (lấy mẫu đều trên toàn bộ khóa) và 1 triệu truy vấn vắng mặt (hoặc n nếu n < 10^6); thời gian lấy trung vị của 5 lượt sau 2 lượt khởi động.
- Lý thuyết dùng để đối chiếu (α = n/M, đếm số ô/nút kiểm tra):
  - SC: có mặt $1 + \alpha/2$; vắng mặt α.
  - LP: có mặt $\frac{1}{2}(1 + 1/(1-\alpha))$; vắng mặt $\frac{1}{2}(1 + 1/(1-\alpha)^2)$.

2. Thí nghiệm A – ảnh hưởng của α ($M = 2^{20}$)

2.1 Số lần dò: đo so với lý thuyết

α	LP có mặt (LT / đo)	LP vắng mặt (LT / đo)	SC có mặt (LT / đo)	SC vắng mặt (LT / đo)
0,50	1,50 / 1,503	2,50 / 2,506	1,250 / 1,250	0,500 / 0,499
0,75	2,50 / 2,507	8,50 / 8,530	1,375 / 1,375	0,750 / 0,748
0,90	5,50 / 5,504	50,5 / 50,07	1,450 / 1,450	0,900 / 0,898
0,95	10,5 / 10,63	200,5 / 204,7	1,475 / 1,476	0,950 / 0,949

Số đo khớp lý thuyết trong khoảng 0,1% (SC) và ⩽ 2,1% (LP, mốc α = 0,95 vắng mặt là lệch lớn nhất). Điều này cho thấy hàm băm Fibonacci trên khóa ngẫu nhiên đều đủ gần "băm ngẫu nhiên đều" để các công thức Knuth áp dụng được.

Một kết quả phụ: số lần dò trung bình khi **chèn** (cũng là số ô kiểm tra trong lượt dựng bảng) của lp-tomb là 1,503 / 2,507 / 5,504 / 10,63, tức băng số lần dò tìm có mặt (đúng như lý thuyết: chi phí chèn khóa thứ i băng chi phí tìm vắng mặt ở tải i/M, và trung bình theo i trùng với tìm có mặt ở tải α). Với SC, chèn mất α/2 nút (0,250 / 0,375 / 0,450 / 0,476) vì chuỗi mới dài dần trong lúc dựng.

2.2 Thời gian (ns/thao tác, trung vị, driver)

α	SC có mặt	LP có mặt	SC vắng mặt	LP vắng mặt	SC dựng bảng	LP dựng bảng
0,50	54,0	37,0	24,7	41,4	59,5	69,9
0,75	39,9	46,2	52,8	57,6	62,1	60,4

α	SC có mặt	LP có mặt	SC vắng mặt	LP vắng mặt	SC dựng bảng	LP dựng bảng
0,90	53,0	59,2	39,2	141,0	57,0	67,1
0,95	50,7	84,0	47,7	301,4	63,2	76,0

Xu hướng rõ: **tìm vắng mặt của LP tăng gấp ~7 lần từ α = 0,5 đến 0,95 (41 → 301 ns)**, bám sát đường $1/(1-\alpha)^2$, còn SC gần như phẳng. Với tìm có mặt LP tăng ~2,3 lần, còn SC phẳng. Ở α thấp (0,5) LP nhanh hơn hoặc bằng SC khi tìm có mặt. Các số của SC dao động ±20–30% giữa các mốc (ví dụ vắng mặt 24,7 ns ở α = 0,5 nhưng 52,8 ns ở α = 0,75), tức là nhiễu do máy ảo; chỉ nên kết luận theo xu hướng.

Lượt time chính thức của Runner (1 lần chạy, chỉ khóa có mặt, ms cho n lượt `get`): SC / LP = 32 / 30 (α = 0,5), 53 / 51 (0,75), **262** / 67 (0,90), 83 / 79 (0,95). Mốc SC α = 0,90 = 262 ms là **ngoại lệ** (cùng mốc, driver đo 53 ns/op ≈ 50 ms cho 943k khóa), nhiều khả năng do GC hoặc nhiễu máy ảo ở một lần chạy đơn lẻ. Khi chạy trên máy lab nên dùng `--repeat ⩾ 5` và lấy trung vị.

3. Thí nghiệm B – ảnh hưởng của M (α = 0,75)

3.1 Số lần dò độc lập với M

M	SC có mặt	SC vắng mặt	LP có mặt	LP vắng mặt
2^{18}	1,377	0,751	2,496	8,363
2^{20}	1,375	0,748	2,507	8,530
2^{21}	1,375	0,749	2,507	8,543
2^{22}	1,375	0,750	2,497	8,504
2^{23}	1,375	0,750	2,502	8,473

Số lần dò **không đổi theo M** và vẫn bám lý thuyết (1,375 / 0,75 / 2,5 / 8,5), chỉ phụ thuộc vào α. Mốc nhỏ nhất 2^{18} lệch nhẹ ở LP vắng mặt (8,36) do phương sai mẫu của bảng nhỏ.

3.2 Thời gian tăng theo M dù số lần dò không đổi

M	SC có mặt	LP có mặt	SC vắng mặt	LP vắng mặt
2^{18}	36,9	35,5	27,1	33,0

M	SC có mặt	LP có mặt	SC vắng mặt	LP vắng mặt
2^{20}	39,9	46,2	52,8	57,6
2^{21}	54,2	63,7	44,9	79,5
2^{22}	66,9	104,0	61,6	91,0
2^{23}	85,7	131,9	66,1	106,2

(ns/thao tác). Từ 2^{18} lên 2^{23} thời gian tăng ~2,3× (SC) và ~3,7× (LP) trong khi số lần dò không đổi. Đây là hiệu ứng **bộ nhớ đệm**: bảng nhỏ nằm gọn trong L2/L3, bảng lớn thì mỗi lần dò là một lần trượt cache. Thời gian Runner chính thức (ms cho n lượt `get`, SC / LP): 21/17 (2^{18}), 53/51 (2^{20}), 75/86 (2^{21}), 143/171 (2^{22}), 403/490 (2^{23}).

Điều đáng chú ý là **LP chậm hơn SC ở M lớn** (131,9 so với 85,7 ns ở 2^{23} khi tìm có mặt) dù ít "nhảy" bộ nhớ hơn trên lý thuyết. Giả thuyết hợp lý nhất (chưa kiểm chứng bằng bộ đếm phần cứng): lp-tomb lưu ba mảng tách rời (`keys`, `values`, `state`) nên mỗi ô dò chạm tới 2 đường cache (state + keys), thêm đường thứ 3 (values) khi tìm thấy; với ~2,5 ô dò, LP chạm ≥ 3 đường cache, còn SC chỉ chạm bucket + 1,375 nút. Cần kiểm chứng trên máy lab (xem mục 7).

4. Bộ dồn cụm ($\alpha = 0{,}75$, 10% khóa có $h(k) < M/64$)

M	Thuật toán	Có mặt (lần dò)	Vắng mặt (lần dò)	Thời gian Runner (ms)
2^{18}	SC	1,610	0,747	22
2^{18}	LP-tomb	**2806**	**6956**	550
2^{20}	SC	1,612	0,750	51
2^{20}	LP-tomb	**11265**	**27803**	11471

- **SC gần như không bị ảnh hưởng**, và kết quả giải thích được bằng lý thuyết: 10% khóa (0,075M khóa) rơi vào M/64 bucket nóng nên chuỗi trong vùng nóng dài ≈ 4,8 + 0,7 nút. Số lần dò có mặt ≈ 0,1 × (1 + 5,5/2) + 0,9 × 1,34 ≈ 1,62 (đo 1,61); số lần dò vắng mặt ≈ (1/64) × 5,475 + (63/64) × 0,675 ≈ 0,750 (đo 0,750).
- **LP sụp đổ**: vùng nóng có 0,075M khóa nhưng chỉ có M/64 ≈ 0,0156M ô gốc nên tạo một cụm sơ cấp khổng lồ tràn ra khắp bảng. Số lần dò tăng **tuyến tính theo M** (×4,0 khi M tăng ×4: 2806 → 11265), tức O(M) thay vì O(1), và thời gian chậm hơn SC ~25× (2^{18}) và

~225× (2^{20}).

- Mốc LP 2^{20} dồn cụm chạy 11,5 s ở lượt time, lớn hơn mọi mốc khác; với timeout mặc định 60 s thì cách cái ngưỡng không xa, nên với bảng lớn hơn hoặc máy chậm hơn mốc này sẽ **timeout**. Hãy ghi rõ trong paper.

5. Đối chiếu `output_check` và tính hợp lệ

- 44 dòng CSV đều `status=ok`, 22 cặp (cùng dataset, M, α, seed, mode) cho sc và lp-tomb: `input_sha256` **trùng** và `output_check` **trùng** ở cả 22/22.
- Lưu ý: vì mọi giá trị đều bằng 1 và mọi khóa đều có mặt, `output_check` luôn bằng n, nên nó chỉ chứng minh "không mất khóa nào", chưa kiểm chứng được giá trị trả về. Nếu muốn kiểm tra chặt hơn, nên đặt value = chỉ số hoặc một hàm của khóa.

6. Bản nháp Discussion – chi phí cấp phát `Node` so với mảng nguyên thủy

Bố cục bộ nhớ. Trên HotSpot với con trỏ nén (heap < 32 GB), một `Node` của SeparateChaining gồm tiêu đề đối tượng 12 B, `long key` 8 B, `int value` 4 B và tham chiếu `next` 4 B, tổng 28 B và được căn lên **32 B**. Mỗi bucket thêm 4 B (tham chiếu). Ngược lại lp-tomb dùng ba mảng nguyên thủy `long[]`, `int[]`, `byte[]` nên mỗi ô tốn 8 + 4 + 1 = **13 B**, không có tiêu đề đối tượng riêng cho từng phần tử. Tính trên mỗi khóa sống:

α	SC: 32 + 4/α (B/khóa)	LP: 13/α (B/khóa)	Tỷ lệ SC/LP
0,50	40,0	26,0	1,5×
0,75	37,3	17,3	2,2×
0,90	36,4	14,4	2,5×
0,95	36,2	13,7	2,6×

Driver đo heap thực (sau `System.gc()`) cho SC ≈ 31,8–38,6 B/khóa, khớp công thức; LP đo được 11,8–24,6 B/khóa, thấp hơn công thức khoảng 1–5 B (ước lượng heap bằng `totalMemory - freeMemory` có sai số vài B/khóa, mốc LP α = 0,75 ở 2^{20} là nhiễu GC rõ nhất). Vì vậy bảng trên nên được xem là số liệu chính, số đo là kiểm tra chéo. Điểm cốt lõi: **chi phí cố định 32 B/Node làm SC tốn gấp 1,5–2,6× bộ nhớ**, và độ chênh lớn dần khi α tăng vì LP "chia" 13 B cho nhiều khóa hơn, còn SC vẫn trả 32 B cho mỗi khóa.

Chi phí cấp phát. Mỗi `put` chèn mới của SC thực hiện một `new Node`. Cấp phát bằng con trỏ trong TLAB rất rẻ, nhưng tổng n đối tượng sống phải được GC đánh dấu và sao chép khi lên old generation, trong khi LP chỉ có 3 đối tượng mảng lớn. Trong phép đo của chúng tôi, thời gian dựng bảng SC và LP **tương đương** (62,1 so với 60,4 ns/khóa ở α = 0,75, M = 2^{20}; SC 57–63, LP 60–76 ns trên cả bốn mốc α), vì phần cấp phát của SC được bù bằng chi phí dò của LP (2,5 → 10,6 ô/lần chèn). Chi phí của `Node` do đó thể hiện chủ yếu ở **bộ nhớ và GC**, chưa thể hiện ở thời gian dựng. Về GC, tổng thời gian dừng GC (không tính `System.gc()` tường minh) cả tiến trình driver ở 2^{23} là 9,3 s với SC so với 6,1 s với LP (+54%), ở 2^{20} là 0,81 s so với 0,75 s (+8%). Con số này gồm cả GC do sinh khóa bằng `HashSet<Long>` nên chỉ phần chênh lệch có ý nghĩa; chênh lệch tăng theo M phù hợp nhận định rằng số đối tượng sống là nguồn chi phí GC.

Truy cập bộ nhớ. SC phải theo con trỏ: bucket → `Node` → `next`; các `Node` cấp phát theo thứ tự chèn nên các nút của cùng một chuỗi nằm rải rác ngẫu nhiên trong heap. LP duyệt các ô liền kề trong mảng nên giữ được tính cục bộ không gian. Tuy nhiên, kết quả đo cho thấy ưu thế lý thuyết của LP **không xuất hiện rõ** ở α = 0,75: SC nhanh hơn LP ở M ⩾ 2^{21} (ví dụ 85,7 so với 131,9 ns ở 2^{23}), vì lp-tomb tách dữ liệu thành ba mảng nên mỗi ô dò chạm nhiều đường cache, và vì số ô dò của LP (2,5 có mặt, 8,5 vắng mặt) lớn hơn số nút của SC (1,375 và 0,75). LP chỉ thật sự vượt SC ở tải thấp (α = 0,5, có mặt: 37,0 so với 54,0 ns) hoặc khi bảng năm gọn trong cache. Ở tải cao (α ⩾ 0,9) LP thua rõ khi tìm vắng mặt (141 và 301 ns so với 39 và 48 ns).

Kết luận cho Discussion. (i) SC đổi bộ nhớ lấy độ ổn định: tốn 1,5–2,6× bộ nhớ nhưng số lần dò và thời gian hầu như không phụ thuộc α và miễn nhiễm với dữ liệu dồn cụm. (ii) LP tiết kiệm bộ nhớ nhờ mảng nguyên thủy nhưng nhạy với α (vắng mặt tăng như $1/(1-\alpha)^2$) và với phân bố khóa (O(M) khi dồn cụm). (iii) Chi phí cấp phát `Node` không làm chậm giai đoạn dựng bảng trong thí nghiệm này mà thể hiện qua bộ nhớ và GC.

7. Hạn chế và việc cần làm trước khi đưa vào paper

1. Chạy lại toàn bộ phần **time** và **heap** trên PC lab với `--machine-id` thật, `--repeat ⩾ 5`, báo cáo trung vị và khoảng dao động. Số lần dò không cần chạy lại.
2. Kiểm tra giả thuyết về 3 mảng và cache bằng `perf stat` (cache-misses) hoặc bằng một biến thể lp-tomb dùng mảng xen kẽ (key, value cùng một `long[]`/struct).
3. Đo GC chuẩn: chạy riêng bước dựng bảng với `-Xlog:gc` và heap nhỏ hơn (ví dụ `-Xmx512m` cho 2^{22}) để thấy áp lực GC của SC rõ hơn; bài này mới đo gián tiếp.
4. Thay bảng lý thuyết ở mục 3 của đề tài nếu khác công thức Knuth tôi đã dùng.

5. `BenchmarkRunner` hiện chỉ đo tìm có mặt. Nếu đề tài yêu cầu số liệu vắng mặt và dựng bảng trong CSV chính thức, cần thêm vào Runner (hiện chỉ có trong `rq1_driver_results.csv`).
6. Nên nâng `output_check` (xem mục 5).

8. Tệp đi kèm

`fig1_probes_vs_alpha.png`, `fig2_time_vs_alpha.png`, `fig3_vs_M.png`, `fig4_clustered.png`, `fig5_memory.png`, `rq1_runner_results.csv` (44 dòng, schema 20 cột), `rq1_driver_results.csv`, `rq1_jobs.txt`.